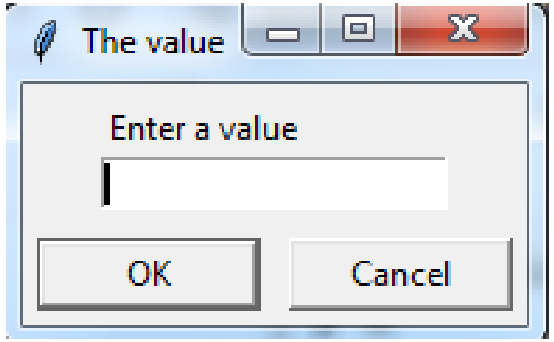
DialogBoxes

- There are different types of Dialog Boxes in tkinter

- Input Dialog Box

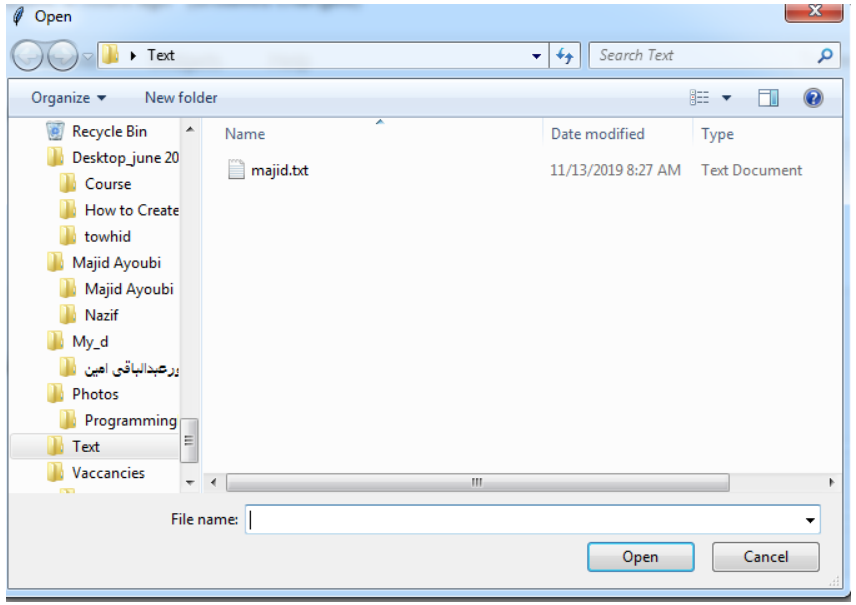
```
from tkinter.simpledialog import *

askinteger("The value","Enter a value")
```



File Dialog

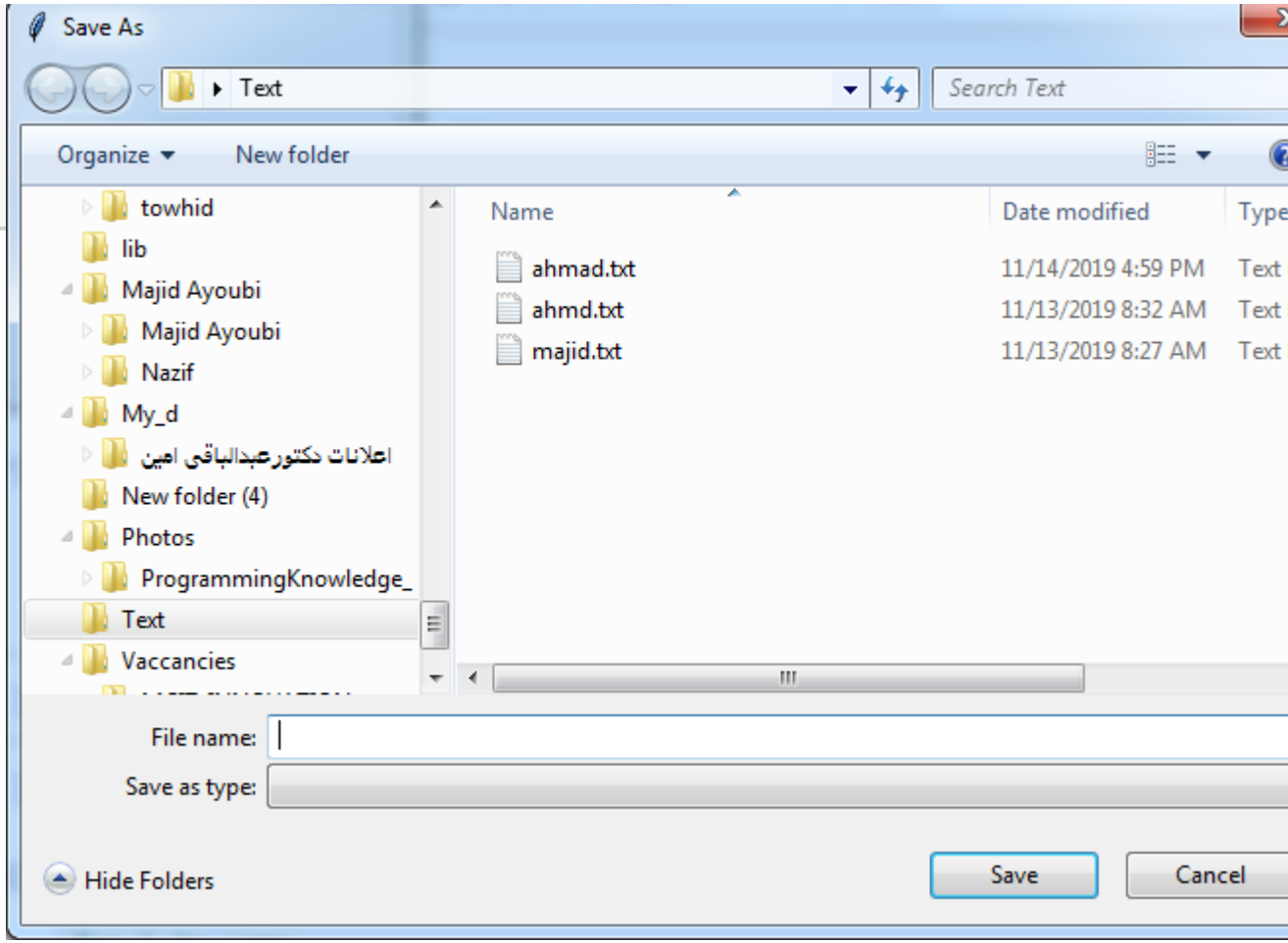
```
from tkinter.filedialog import *

askopenfilename()
```

```
'C:/Users/Majid/Desktop/Text/majid.txt'
```



Save file dialog

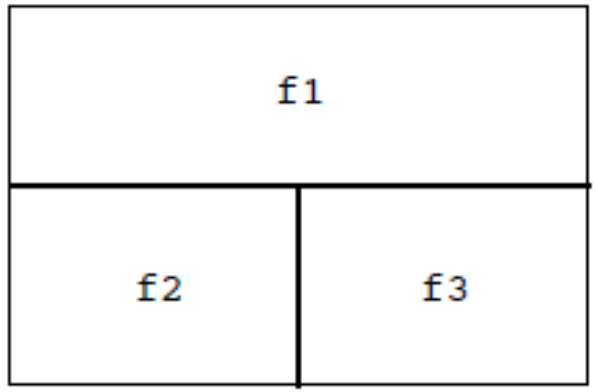
```
from tkinter.filedialog import *

asksaveasfile(mode='w', defaultextension='.txt')
```



Frames

- Typically used to subdivide a window into logical components.
- Widgets are then placed into each frame
- Frame is used as the "parent" window

```
frame1=Frame(root)
```



Adding widgets into frame

```
but1=Radiobutton(frame1, text='Python', variable=var, value='Python')
but1.grid(row=0, column=0)

but2=Radiobutton(frame1, text='Java', variable=var, value='Java')
but2.grid(row=1, column=0)

but3=Radiobutton(frame1, text='C++', variable=var, value='C++')
but3.grid(row=2, column=0)
```

Thank You!

Data Viaualization

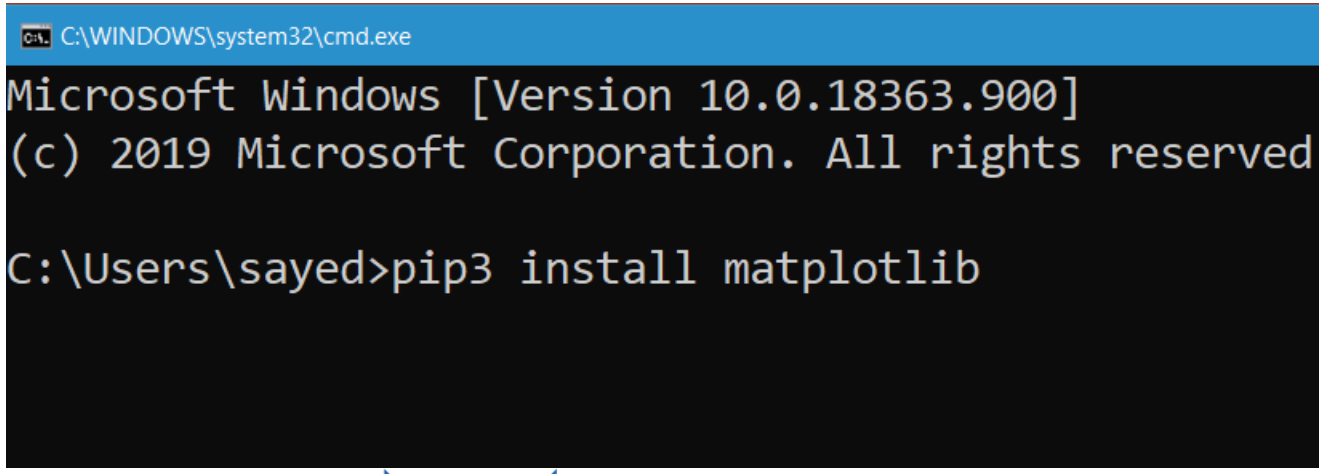
- Data visualization is the discipline of trying to understand data by placing it in a visual context so that patterns, trends, and correlations that might not otherwise be detected can be exposed.
- Python offers multiple great graphing libraries packed with lots of different features. Whether you want to create interactive or highly customized plots, Python has an excellent library for you.

Python Data Visualization

- Data visualization is the discipline of trying to understand data by placing it in a visual context so that patterns, trends and correlations that might not otherwise be detected can be exposed.
- Python offers multiple great graphing libraries that come packed with lots of different features. No matter if you want to create interactive, live or highly customized plots python has an excellent library for you.
- When we present data graphically, we can see the patterns and insights we're looking for. It becomes easier to grasp difficult concepts or identify new trends we may have missed. We can use data visualizations to make an argument, or to support a hypothesis, or to explore our world in different ways.

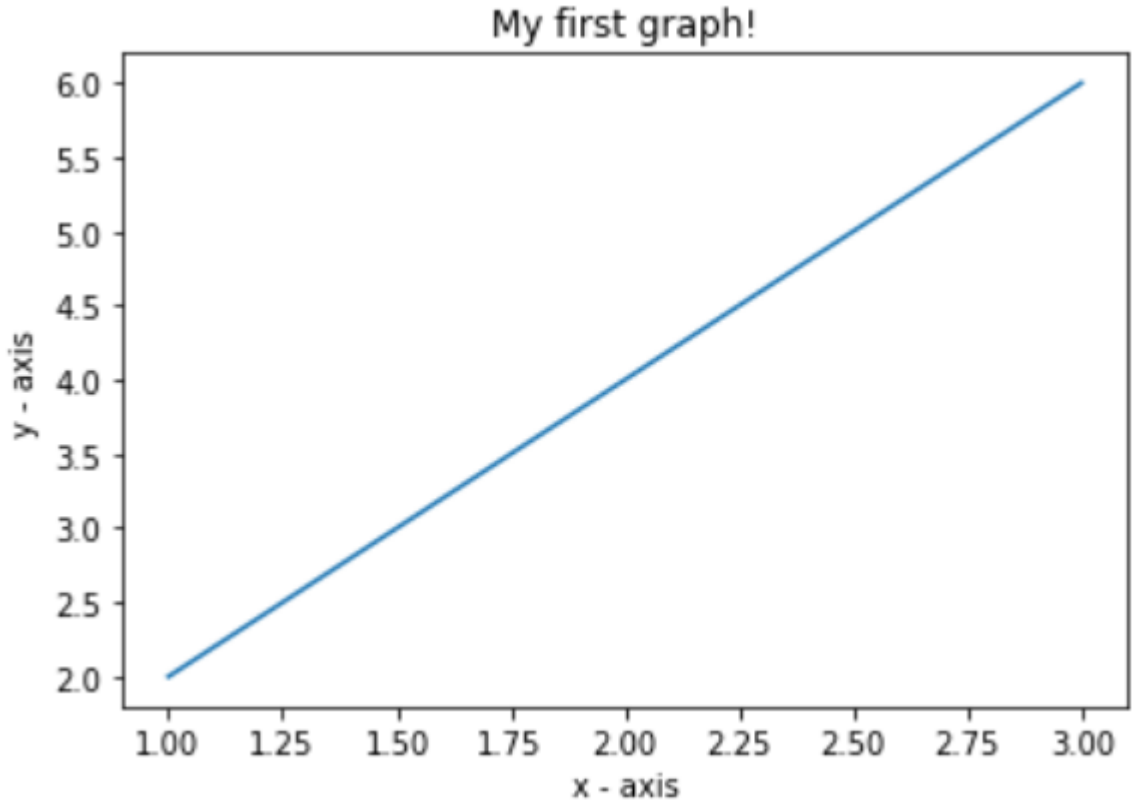
Matplotlib

- We will learn Plotting and charting in python with Matplotlib Library,
- Matplotlib is arguably the most popular graphing and data visualization library for Python.
- To use this library, we may need to install it on the computer:
- pip3 install matplotlib / python -m pip install -U matplotlib



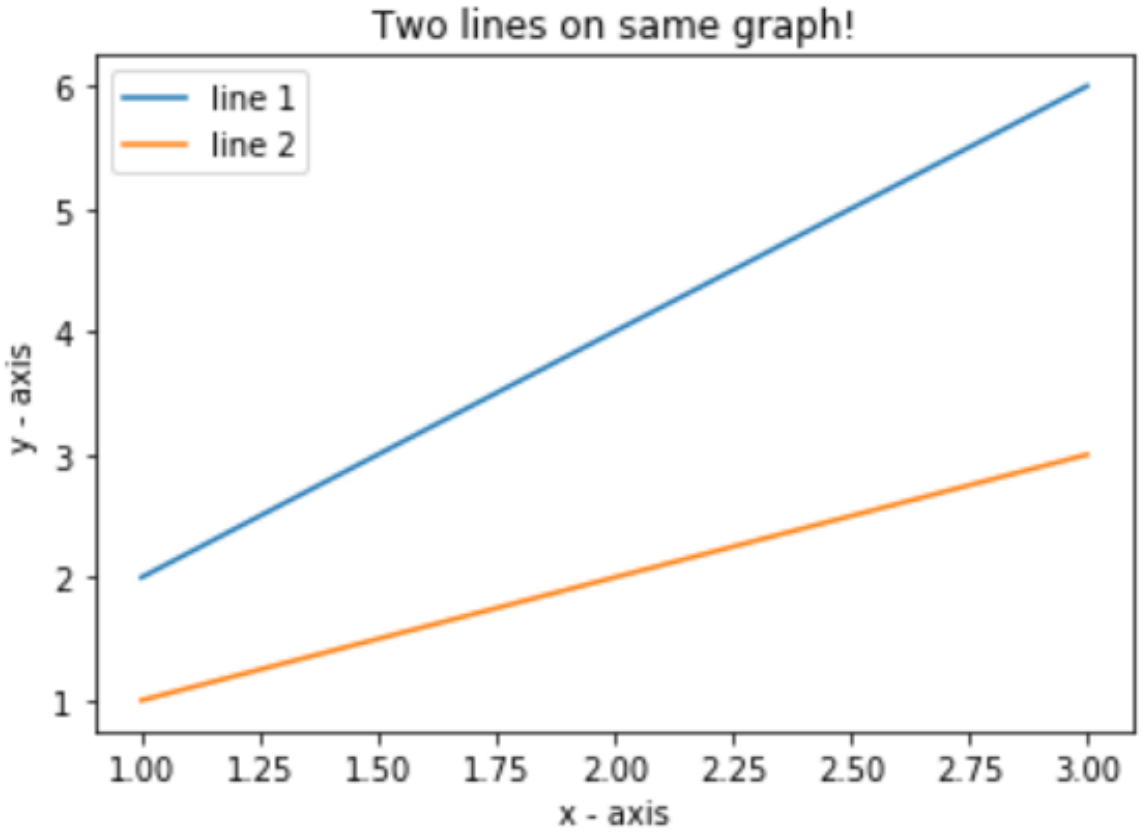
Plotting one line

```
1  import matplotlib.pyplot as plt
2
3  x = [1,2,3]
4  y = [2,4,6]
5
6  plt.plot(x, y)
7
8  plt.xlabel('x - axis')
9  plt.ylabel('y - axis')
10
11 plt.title('My first graph!')
12
13 plt.show()
```



Plotting two lines

```python
import matplotlib.pyplot as plt

x1 = [1,2,3]
y1 = [2,4,6]
plt.plot(x1, y1, label = "line 1")

x2 = [1,2,3]
y2 = [1,2,3]
plt.plot(x2, y2, label = "line 2")

plt.xlabel('x - axis')
plt.ylabel('y - axis')
plt.title('Two lines on same graph!')
plt.legend()
plt.show()
```



Customizing the plot

```
1  import matplotlib.pyplot as plt
2
3  x = [1,2,3,4,5,6]
4  y = [2,4,1,5,2,6]
5
6  plt.plot(x, y, color='green', linestyle='dashed', linewidth = 3,
7           marker='o', markerfacecolor='blue', markersize=12)
8
9  plt.ylim(1,8)
10 plt.xlim(1,8)
11
12 plt.xlabel('x - axis')
13 plt.ylabel('y - axis')
14
15 plt.title('Some cool customizations!')
16 plt.show()
17 %matplotlib inline
```